



Sequence 4.5 – IRBuilder

P. de Oliveira Castro S. Tardieu

Creating an IR of the program

How to build an IR representation of our program?

Instead of writing IR directly, we call a programmatic API, the *IR Builder*.

Advantages:

- Faster: IR is directly built in memory
- Robust: The API enforces many legality rules of the IR
- Cleaner: The IR Builder offers high-level abstractions for building the IR

- An IR Builder keeps track of an *insert point*. New instructions are added after the insert point which is then automatically moved forward.
- High level builders for complex patterns such as:
 - Calling multi-parameters functions
 - Accessing the fields of a structure
 - Creating conditional branches

Context and Function

- A Builder operates in a given *Context*
 - The *Context* captures the global data of a compilation unit
 - Whenever the builder creates a new global variable, global type, or function declaration, it is added to the *Context*
- A Builder inserts instructions in a given *BasicBlock*
 - A *BasicBlock* operates within a *Context* and belongs to a *Function*

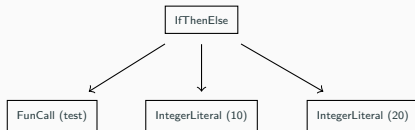
Inserting new instructions

How to translate $(10+5)*2$ in IR?

```
1  llvm::IRBuilder Builder(Context);
2  [...]
3  llvm::BasicBlock *const body =
4      llvm::BasicBlock::Create(Context, "body", current_function);
5
6  Builder.SetInsertPoint(body);
7
8  llvm::Value * a =
9      Builder.CreateAdd(Builder.getInt32(10), Builder.getInt32(5));
10 llvm::Value * b =
11     Builder.CreateMul(a, Builder.getInt32(2));
```

How to translate Tiger AST to LLVM IR

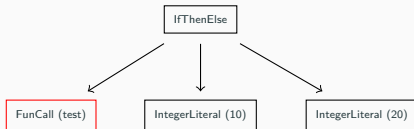
```
if test() then 10 else 20
```



Translate with a visitor that returns LLVM values.

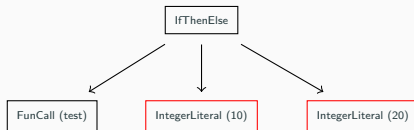
FunCall

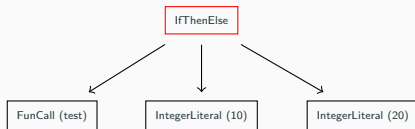
```
1 // Simplified (no static link + no arguments)
2 llvm::Value *IRGenerator::visit(const FunCall &call) {
3     const FunDecl &decl = call.get_decl().get();
4     llvm::Function *callee =
5         Module->getFunction(decl.get_external_name().get());
6     return Builder.CreateCall(callee, {}, "call");
7 }
```



IntegerLiteral

```
1  llvm::Value *IRGenerator::visit(const IntegerLiteral &literal) {  
2      return Builder.getInt32(literal.value);  
3  }
```





`IfThenElse` is more complex: diverging control requires multiple basic blocks. To simplify, in the following we assume that the `if` always returns a value.

IfThenElse: Prolog

```
1  llvm::Value *IRGenerator::visit(const IfThenElse &ite) {
2
3      // We create an allocation in the function entry block
4      // to store the if result (see lecture 4.4)
5      llvm::Value *const result =
6          alloca_in_entry(llvm_type(ite.get_type()), "if_result");
7
8      // We create three empty basic blocks
9      llvm::BasicBlock *const then_block =
10         llvm::BasicBlock::Create(Context, "if_then", current_function);
11      llvm::BasicBlock *const else_block =
12         llvm::BasicBlock::Create(Context, "if_else", current_function);
13      llvm::BasicBlock *const end_block =
14         llvm::BasicBlock::Create(Context, "if_end", current_function);
```

IfThenElse: Condition

We branch depending on the condition,

```
Builder.CreateCondBr(  
    Builder.CreateIsNull(ite.get_condition().accept(*this)),  
    then_block,  
    else_block);
```

`ite.get_condition().accept(*this)` returns the result LLVM Value of the `FunCall test()` translation.

IfThenElse: Then and Else bodies

```
llvm::Value *const result = alloca_in_entry(...);
```

Block for the then part:

```
1  Builder.SetInsertPoint(then_block);  
2  llvm::Value *const then_result =  
3      ite.get_then_part().accept(*this);  
4  Builder.CreateStore(then_result, result);  
5  Builder.CreateBr(end_block);
```

Block for the else part:

```
1  Builder.SetInsertPoint(else_block);  
2  llvm::Value *const else_result =  
3      ite.get_else_part().accept(*this);  
4  Builder.CreateStore(else_result, result);  
5  Builder.CreateBr(end_block);
```

IfThenElse: Epilog

```
llvm::Value *const result = alloca_in_entry(...);
```

Block for joining the then and else parts:

```
1  Builder.SetInsertPoint(end_block);  
2  return Builder.CreateLoad(llvm_type(ite.get_type()), result);
```